

Deploy java applications

Java Archive (Jar) files

- fundamental packaging format in Java.
- used to **aggregate and compress multiple files into a single archive**,
- distribute and deploy Java applications.

<https://naveen-metta.medium.com/demystifying-jar-fat-jar-war-and-ear-files-in-java-d6802af9beda>

JAR file

- simply ZIP files that contain all of the resources required to run a Java application.
- A JAR file typically contains:
 - Java source files
 - Property files
 - XML files
 - Image files
 - A manifest file

<https://www.theserverside.com/blog/Coffee-Talk-Java-News-Stories-and-Opinions/Run-JAR-file-example-windows-linux-ubuntu>

Manifest.mf

- Text file located in the jar META-INF/MANIFEST.MF

Manifest-Version: 1.0

Created-By: 1.7.0_06 (Oracle Corporation)

Main-Class: MyPackage.MyClass

- Can define which class will be executed

```
java -jar MyJar.jar
```

- It will exec MyPackage.Myclass within MyJar.java

<https://docs.oracle.com/javase/tutorial/deployment/jar/manifestindex.html>

Manifest.mf

- Text file located in the jar META-INF/MANIFEST.MF

Manifest-Version: 1.0

Created-By: 1.7.0_06 (Oracle Corporation)

Main-Class: **MyPackage.MyClass**

- Can define which class will be executed

```
java -jar MyJar.jar
```

- It will exec **MyPackage.Myclass** within MyJar.java

<https://docs.oracle.com/javase/tutorial/deployment/jar/manifestindex.html>

If I want to run a specific class?

and if I want to run a specific class within the jar ?

Great question! If you want to run a **specific class within a JAR**, not necessarily the one defined as the **Main-Class** in the manifest, you can do it like this:

✓ Command to Run a Specific Class from a JAR

bash

Copy

Edit

```
java -cp your-jar-file.jar your.package.ClassName
```



- **-cp** (or **-classpath**) tells Java to use your JAR for classes.
- Replace **your-jar-file.jar** with the name of your JAR.
- Replace **your.package.ClassName** with the **fully qualified name** of the class you want to run.



Creating a jar manually

Creating a Jar File:

To create a basic Jar file, you can use the following command:

```
jar cf MyJar.jar -C /path/to/classes .
```

More details on jar command

<https://docs.oracle.com/javase/tutorial/deployment/jar/manifestindex.html>

<https://joshiadvait8.medium.com/how-to-create-jar-file-easy-way-of-jar-from-java-files-b8c916ab291e>

Jar commands: some examples

- Run a JAR file
 - `jar -jar jarfilename.jar`
- Create a JAR file
 - `jar cf jarfiletocreate.jar file-list`
- Open a JAR file
 - `jar xf jar-file-to-open.jar`
- Run a JAR without a manifest
 - `java -cp jarfilename.jar com.exe`

More details on jar command

<https://docs.oracle.com/javase/tutorial/deployment/jar/manifestindex.html>

<https://joshiadvait8.medium.com/how-to-create-jar-file-easy-way-of-jar-from-java-files-b8c916ab291e>

Asked the chatGPT...

can you give the command line to run the class within the jar

Certainly! To run a Java class from within a **JAR file** that uses an embedded Jetty server, you need to:

1. **Package your project into a JAR** with all dependencies included.
2. **Ensure the JAR has a manifest file** that specifies the main class.
3. **Run the JAR** using the `java -jar` command.



Asked the chatGPT...

can you give the command line to run the class within the jar

Certainly! To run a Java class from within a **JAR file** that uses an embedded Jetty server, you need to:

1. Package your project into a JAR with all dependencies included.
2. Ensure the JAR has a manifest file that specifies the main class.
3. Run the JAR using the `java -jar` command.



Dependencies in java

- External Java libraries required for Project
 - Can be jars
 - Managed by user
- Must be within execution classpath and or Jar
- If not we get ClassNotFoundException

**** Problem scenario #2: ClassLoader.loadClass() ****

Missing Java class

```
java.lang.ClassNotFoundException: org.ph.javaee.training5.ClassB
    at java.net.URLClassLoader$1.run(URLClassLoader.java:366)
    at java.net.URLClassLoader$1.run(URLClassLoader.java:355)
    at java.security.AccessController.doPrivileged(Native Method)
    at java.net.URLClassLoader.findClass(URLClassLoader.java:354)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:423)
    at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:308)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:356)
    at org.ph.javaee.training5.ClassNotFoundExceptionSimulator.main(Cl
```

Caller or referencing class

Dependencies in java

- External Java libraries required for Project
 - Can be jars
 - Managed by user
- **Must be within execution classpath** and or Jar
- If not we get ClassNotFoundException

```
** Problem scenario #2: ClassLoader.loadClass() **
```

Missing Java class

```
java.lang.ClassNotFoundException: org.ph.javaee.training5.ClassB
    at java.net.URLClassLoader$1.run(URLClassLoader.java:366)
    at java.net.URLClassLoader$1.run(URLClassLoader.java:355)
    at java.security.AccessController.doPrivileged(Native Method)
    at java.net.URLClassLoader.findClass(URLClassLoader.java:354)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:423)
    at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:308)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:356)
    at org.ph.javaee.training5.ClassNotFoundExceptionSimulator.main(Cl
```

Caller or referencing class

Dependencies in java

- External Java libraries required for Project
 - Can be jars
 - Managed by user
- Must be within execution classpath and **or Jar**
- If not we get ClassNotFoundException

**** Problem scenario #2: ClassLoader.loadClass() ****

Missing Java class

```
java.lang.ClassNotFoundException: org.ph.javaee.training5.ClassB
    at java.net.URLClassLoader$1.run(URLClassLoader.java:366)
    at java.net.URLClassLoader$1.run(URLClassLoader.java:355)
    at java.security.AccessController.doPrivileged(Native Method)
    at java.net.URLClassLoader.findClass(URLClassLoader.java:354)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:423)
    at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:308)
    at java.lang.ClassLoader.loadClass(ClassLoader.java:356)
    at org.ph.javaee.training5.ClassNotFoundExceptionSimulator.main(Cl
```

Caller or referencing class

In default java projects in VSCode

- Place dependencies jar in
 - <project>/libs (default)
 - Path within settings.json field
 - "java.project.referencedLibraries"



Create jar with dependencies

Step-by-Step Instructions

1. Package into a **fat JAR** (with dependencies)

run

2. Run the JAR

```
bash
```

[Copy](#)[Edit](#)

```
java -jar target/your-artifact-name-jar-with-dependencies.jar
```

and if I want to run a specific class?

and if I want to run a specific class within the jar ?

Great question! If you want to run a **specific class within a JAR**, not necessarily the one defined as the **Main-Class** in the manifest, you can do it like this:

✓ Command to Run a Specific Class from a JAR

bash

Copy

Edit

```
java -cp your-jar-file.jar your.package.ClassName
```



- **-cp** (or **-classpath**) tells Java to use your JAR for classes.
- Replace **your-jar-file.jar** with the name of your JAR.
- Replace **your.package.ClassName** with the **fully qualified name** of the class you want to run.



and if I want to run a specific class?

and if I want to run a specific class within the jar ?

Great question! If you want to run a **specific class within a JAR**, not necessarily the one defined as the **Main-Class** in the manifest, you can do it like this:

✓ Command to Run a Specific Class from a JAR

bash

Copy

Edit

```
java -cp your-jar-file.jar your.package.ClassName
```

- **-cp** (or **-classpath**) tells Java to use your JAR for classes.
- Replace **your-jar-file.jar** with the name of your JAR.
- Replace **your.package.ClassName** with the **fully qualified name** of the class you want to run.

If your jar does not have all dependencies...

If the class requires external dependencies (like Jetty), and your JAR is **not a fat JAR**, then you'll need to add those dependencies to the classpath too, like this:

```
java -cp "myserver.jar:lib/*" com.example.EmbeddedJettyExample
```

The jar with your code ...

```
'myserver.jar':
```

directory(ies) containing all
dependencies that are OUTSIDE the jar

```
lib/*
```

If your jar does not have all dependencies...

If the class requires external dependencies (like Jetty), and your JAR is **not a fat JAR**, then you'll need to add those dependencies to the classpath too, like this:

```
java -cp "myserver.jar:lib/*" com.example.EmbeddedJettyExample
```

On Windows, use `;` instead of `::`:

```
bash
```

```
java -cp "myserver.jar;lib/*" com.example.EmbeddedJettyExample
```

If your jar does not have all dependencies...

If the class requires external dependencies (like Jetty), and your JAR is **not a fat JAR**, then you'll need to add those dependencies to the classpath too, like this:

```
java -cp "myserver.jar:lib/*" com.example.EmbeddedJettyExample
```

The jar with your code ...

`myserver.jar`

directory(ies) containing all dependencies that are OUTSIDE the jar

`lib/*`

Create your Java JAR using the command line



Luiz Gustavo De O. Costa

Following ▾

3 min read · Jan 27, 2022



1



<https://luizcostatech.medium.com/create-your-java-jar-using-the-command-line-efb0fcd6d886>

Two java code file in the directory

```
import java.time.LocalDateTime;
public class MyExecutable {
    public static void main(String... args) {
        System.out.println("Hello there at "+LocalDateTime.now());
    }
}
```

```
public class MyHello {
    public static void main(String... args) {
        System.out.println("Hello");
    }
}
```

Compile the code

```
// compile all *.java files into *.class  
javac .\*.java
```

Create the jar

```
// create a jar with all class in directory  
// and ensure that MyExecutable main is called by  
default
```

```
jar cfe MyJar.jar MyExecutable *.class
```

c: Create the archive.

f: Specify the filename (MyJar.jar).

e: Specify the **Entry point** (MyExecutable).

Create the jar and execute

```
// create a jar with all class in directory
```

```
// and ensure the default
```

 MyJar

CREATES THE JAR WITH THE
MANIFEST

```
jar cfe MyJar.jar
```

 META-INF

ass

MANIFEST.MF

```
Manifest-Version: 1.0  
Created-By: 17.0.14 (Eclipse Adoptium)  
Main-Class: MyExecutable
```

And execute

```
// executes the MyJar  
// calling by default the MyExecutable.class  
java -jar .\MyJar.jar
```

```
Hello there at 2026-05-10T18:42:35.489360200
```

```
// to execute MyHello  
java -cp MyJar.jar MyHello
```

```
Hello
```

You want “myHello” to be the default

```
// Use the m (manifest) flag to tell the tool to use your custom file  
jar cvfm MyApp.jar manifest.txt MyExecutable.class  
MyHello.class
```

```
java -jar .\MyApp.jar
```

Hello

manifest.txt

```
Manifest-Version: 1.0  
Created-By: 17.0.14 (Eclipse Adoptium)  
Main-Class: MyHello
```

When distributing executables

- Make sure that all dependencies are
 - In distributed jars and classes
 - Or in runtime CLASSPATH
 - Or known so the user can add them to classpath

Create jar with dependencies (not yours)

Step-by-Step Instructions

1. Package into a **fat JAR** (with dependencies)

Fat Jar Files

- include **all dependencies within the archive.**
 - self-contained approach
 - simplifies deployment,
 - as the application can be run **without relying on external libraries.**
- Also referred as Uber Jars or executable Jars,

Fat Jar Files

- include **all dependencies within the archive**.
 - self-contained approach
 - simplifies deployment,
 - as the application can be run **without relying on external libraries**.
- Also referred as Uber Jars or executable Jars,

Creating a Fat Jar File:

Using tools like **Maven** or **Gradle**, you can generate a Fat Jar easily.

Fat Jar Files

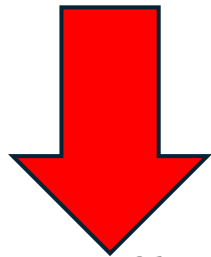
- include **all dependencies within the archive**.
 - self-contained approach
 - simplifies deployment,
 - as the application can be run **without relying on external libraries**.
- Also referred as Uber Jars or executable Jars,

Creating a Fat Jar File:

Using tools like **Maven** or **Gradle**, you can generate a Fat Jar easily.

Fat Jar Files

- include **all dependencies within the archive.**
 - self-contained approach
 - simplifies deployment,
 - as the application can be run **without relying on external libraries.**
- Also referred as Uber Jars or executable Jars,



Creating a Fat Jar File:

Using tools like Maven or Gradle, you can generate a Fat Jar easily.

MavenTM

NEXT